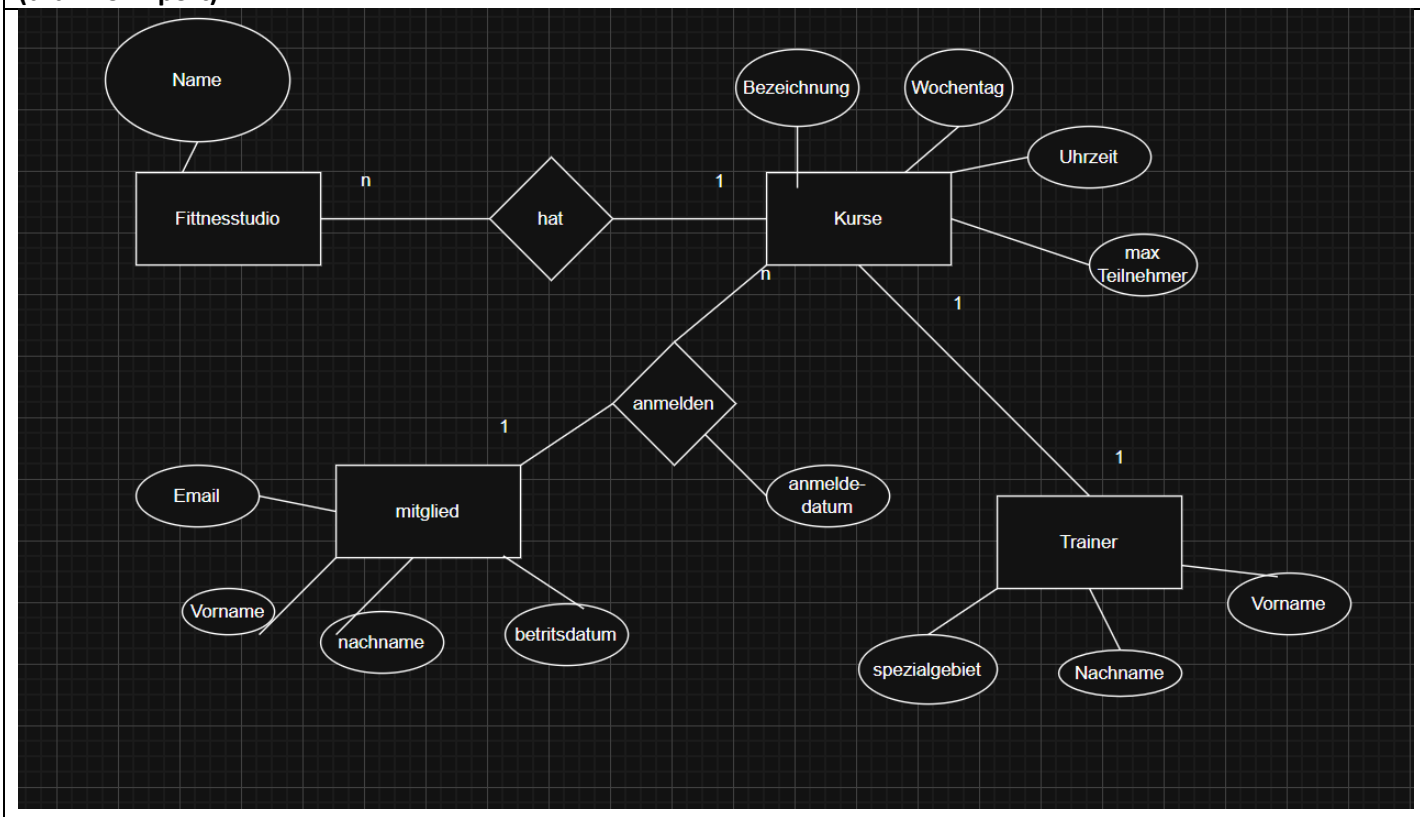
Name: Muhammet Altuntas **Datum:** 08.04.2026 **Klasse:** 3AHIF

3. Grafische Leistungsfeststellung – DBI1
Thema: Fitnessstudio-Verwaltung (ERM, RM, Python/SQLite3, Anvil)

Punkte	**/ 100**				
Note	100-87.5 □ Sehr gut	< 87.5-75 □ Gut	< 75-62.5 □ Befriedigend	< 62.5-50 □ Genügend	< 50 □ Nicht Genügend
Unterschrift					
GitHub-Link	https://github.com/oreca61/GLF3				
Anvil Publish-Link	https://ringed-prickly-noise.anvil.app				

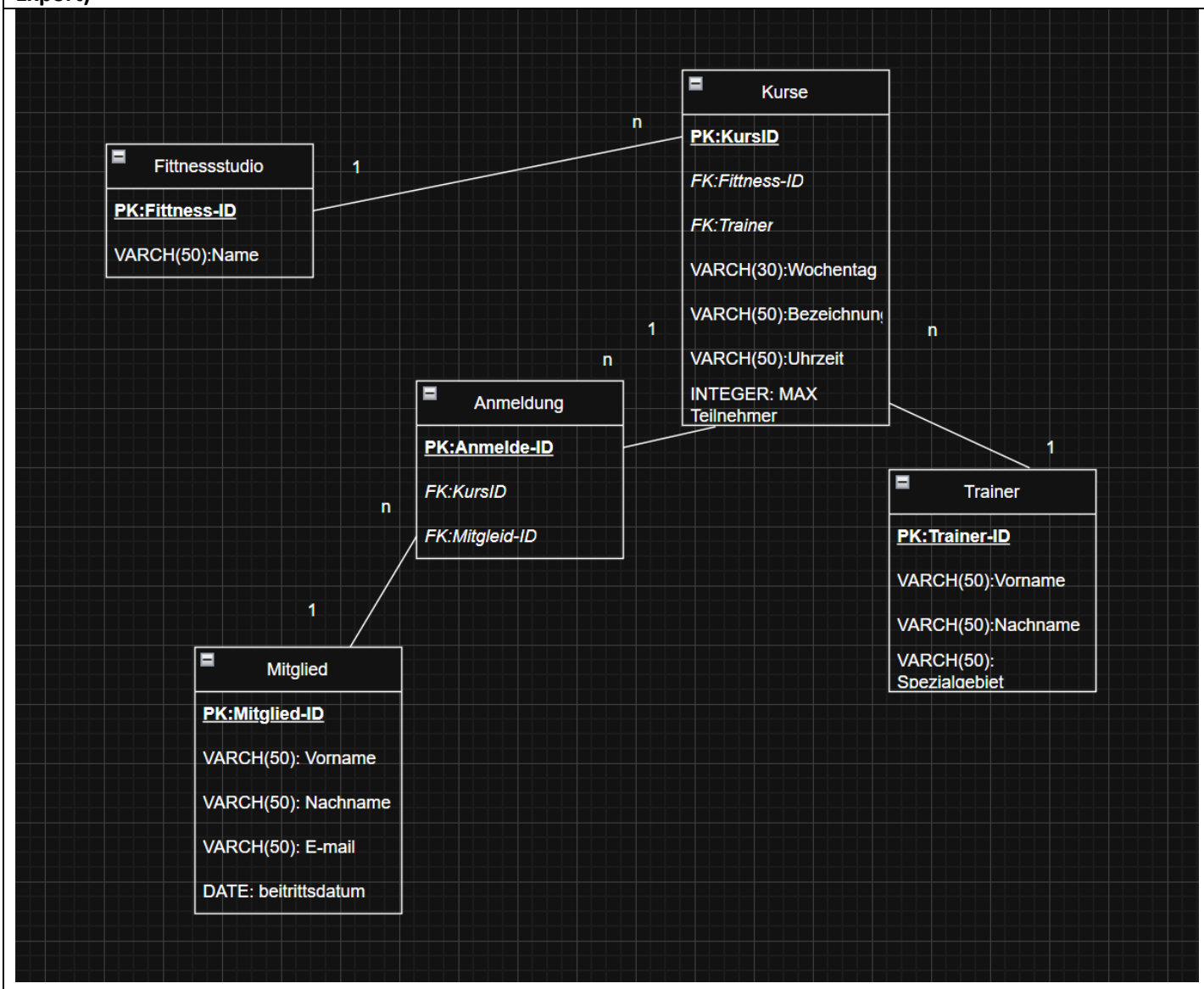
Form	**Pun**
1. Form der Arbeit (Leserlich, vollständige Abgabe, sinnvolle Dateinamen, Ausführbar, Fehlerbehandlung)	
Szenario: Fitnessstudio-Verwaltung	
Ein lokales Fitnessstudio möchte seine Mitgliederverwaltung und Kursbuchungen digitalisieren. Es gelten folgende Rahmenbedingungen: • Das Fitnessstudio bietet verschiedene Kurse an (z. B. Yoga, Pilates, Spinning, Zumba). Von jedem Kurs werden Bezeichnung, Wochentag, Uhrzeit und maximale Teilnehmeranzahl gespeichert. • Jeder Kurs wird von genau einem Trainer geleitet. Ein Trainer kann mehrere Kurse leiten. Von jedem Trainer werden Vorname, Nachname und Spezialgebiet erfasst. • Mitglieder können sich für beliebig viele Kurse anmelden. Für jeden Kurs können sich mehrere Mitglieder einschreiben. Bei jeder Anmeldung wird das Anmeldedatum gespeichert. Von jedem Mitglied werden Vorname, Nachname, E-Mail-Adresse und Beitrittsdatum erfasst.	
1.1. Aufgabe 1 – Entity-Relationship-Modell (ERM)	**Pun**
Erstelle auf Basis des Szenarios ein vollständiges ERM: Füge das fertige ERM als Bild in dieses Dokument ein. **(draw.io-Export)**	___



1.2. Aufgabe 2 – Relationales Modell (RM)	**Pun**

Name: Muhammet Altuntas **Datum:** 08.04.2026 **Klasse:** 3AHIF

Leite aus deinem ERM ein vollständiges Relationales Modell ab: Füge das fertige RM als Bild hier ein. **(draw.io-Export)**



1.3. Aufgabe 3 – Python-Skript mit SQLite3 ★ KI-Tools erlaubt

Pun

Schreibe ein Python-Skript Nachname_Vorname_fitnessstudio.py, das:

1. Eine SQLite-Datenbank Nachname_Vorname_fitnessstudio.db erstellt
2. Alle Tabellen aus dem RM anlegt
3. Sinnvolle Beispieldaten einfügt:
   ≥ 3 Trainer • ≥ 5 Kurse • ≥ 6 Mitglieder • ≥ 8 Anmeldungen

1.4. Aufgabe 4 – Anvil-Implementierung

Pun

Implementiere die Fitnessstudio-Verwaltung als Anvil-Webanwendung (anvil.works).

Pflichtfunktionen:

1. Kursübersicht – Alle Kurse inkl. Trainername

Kurs	Wochentag	Uhrzeit	Trainer	Teilnehmer	Anmelden
Morning Yoga	Montag	08:00	Berger Anna	2/15	Anmelden
Power Lifting	Dienstag	17:00	Huber Markus	2/10	Anmelden
Pilates Basics	Mittwoch	10:00	Mayer Sophie	2/12	Anmelden
HIIT Cardio	Donnerstag	18:30	Schneider Leon	1/20	Anmelden
Abend Yoga	Freitag	19:00	Berger Anna	2/15	Anmelden
Rückenfit	Samstag	09:00	Huber Markus	1/10	Anmelden

2. Neue Anmeldung

Name: Muhammet Altuntas **Datum:** 08.04.2026 **Klasse:** 3AHIF

Mitglied	Anmelden
Bauer Thomas	Anmelden
Fuchs Stefan	Anmelden
Schwarz Maria	Anmelden
Weiß David	Anmelden
Braun Nina	Anmelden

Bonusfunktionen (optional):

- Mitglieder in einem Kurs anzeigen

1.5. Aufgabe 5 - Abgabe

1. Nachname Vorname in der Kopfzeile eintragen
2. GitHub-Link oben eintragen
3. Anvil Publish-Link oben eintragen
4. Folgende Dateien ins Git-Repo pushen
    a. Nachname_Vorname_ERM.draw.io
    b. Nachname_Vorname_RM.draw.io
    c. Nachname_Vorname_fitnessstudio.py
5. Code hier einfügen

Nachname_Vorname_fitnessstudio.py

```
import sqlite3


DB_NAME = "Nachname_Vorname_fitnessstudio.db"


def create_connection():
    conn = sqlite3.connect(DB_NAME)
    conn.execute("PRAGMA foreign_keys = ON;")
    return conn


def create_tables(conn):
    cursor = conn.cursor()

    # Vorhandene Tabellen löschen, damit das Skript mehrfach ausführbar ist
    cursor.execute("DROP TABLE IF EXISTS anmeldung;")
    cursor.execute("DROP TABLE IF EXISTS kurse;")
    cursor.execute("DROP TABLE IF EXISTS mitglied;")
    cursor.execute("DROP TABLE IF EXISTS trainer;")
    cursor.execute("DROP TABLE IF EXISTS fitnessstudio;")

    # Tabelle Fitnessstudio
    cursor.execute("""
```

```
    CREATE TABLE fitnessstudio (
        fitness_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL
    );
""")

# Tabelle Trainer
cursor.execute("""
    CREATE TABLE trainer (
        trainer_id INTEGER PRIMARY KEY AUTOINCREMENT,
        vorname TEXT NOT NULL,
        nachname TEXT NOT NULL,
        spezialgebiet TEXT
    );
""")

# Tabelle Mitglied
cursor.execute("""
    CREATE TABLE mitglied (
        mitglied_id INTEGER PRIMARY KEY AUTOINCREMENT,
        vorname TEXT NOT NULL,
        nachname TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        beitrittsdatum DATE NOT NULL
    );
""")

# Tabelle Kurse
cursor.execute("""
    CREATE TABLE kurse (
        kurs_id INTEGER PRIMARY KEY AUTOINCREMENT,
        fitness_id INTEGER NOT NULL,
        trainer_id INTEGER NOT NULL,
        wochentag TEXT NOT NULL,
        bezeichnung TEXT NOT NULL,
```

```
        uhrzeit TEXT NOT NULL,
        max_teilnehmer INTEGER NOT NULL,
        FOREIGN KEY (fitness_id) REFERENCES fitnessstudio(fitness_id),
        FOREIGN KEY (trainer_id) REFERENCES trainer(trainer_id)
    );
""")

# Tabelle Anmeldung
cursor.execute("""
    CREATE TABLE anmeldung (
        anmelde_id INTEGER PRIMARY KEY AUTOINCREMENT,
        kurs_id INTEGER NOT NULL,
        mitglied_id INTEGER NOT NULL,
        FOREIGN KEY (kurs_id) REFERENCES kurse(kurs_id),
        FOREIGN KEY (mitglied_id) REFERENCES mitglied(mitglied_id)
    );
""")

conn.commit()

def insert_sample_data(conn):
    cursor = conn.cursor()

    # 1 Fitnessstudio
    cursor.execute("""
        INSERT INTO fitnessstudio (name)
        VALUES (?)
    """, ("FitLife Studio",))

    # Mindestens 3 Trainer
    trainer_daten = [
        ("Anna", "Huber", "Yoga"),
        ("Markus", "Leitner", "Krafttraining"),
        ("Sophie", "Gruber", "Cardio"),
    ]
```

```
cursor.executemany("""
    INSERT INTO trainer (vorname, nachname, spezialgebiet)
    VALUES (?, ?, ?)
""", trainer_daten)

# Mindestens 6 Mitglieder
mitglied_daten = [
    ("Lena", "Maier", "lena.maier@example.com", "2024-01-15"),
    ("Paul", "Berger", "paul.berger@example.com", "2024-02-10"),
    ("Mia", "Schwarz", "mia.schwarz@example.com", "2024-03-05"),
    ("Jonas", "Hofer", "jonas.hofer@example.com", "2024-04-20"),
    ("Emma", "Winkler", "emma.winkler@example.com", "2024-05-12"),
    ("Noah", "Fuchs", "noah.fuchs@example.com", "2024-06-01"),
]
cursor.executemany("""
    INSERT INTO mitglied (vorname, nachname, email, beitrittsdatum)
    VALUES (?, ?, ?, ?)
""", mitglied_daten)

# Mindestens 5 Kurse
kurs_daten = [
    (1, 1, "Montag", "Yoga Basic", "2025-05-12 09:00:00", 15),
    (1, 2, "Dienstag", "Power Workout", "2025-05-13 18:00:00", 12),
    (1, 3, "Mittwoch", "Cardio Blast", "2025-05-14 17:30:00", 20),
    (1, 1, "Donnerstag", "Stretch & Relax", "2025-05-15 10:00:00", 10),
    (1, 2, "Freitag", "Muskelaufbau", "2025-05-16 16:00:00", 14),
]
cursor.executemany("""
    INSERT INTO kurse (fitness_id, trainer_id, wochentag, bezeichnung, uhrzeit, max_teilnehmer)
    VALUES (?, ?, ?, ?, ?, ?)
""", kurs_daten)

# Mindestens 8 Anmeldungen
anmeldungen = [
    (1, 1),
```

```
        (1, 2),
        (2, 3),
        (2, 4),
        (3, 5),
        (3, 6),
        (4, 1),
        (5, 2),
        (5, 3),
    ]
    cursor.executemany("""
        INSERT INTO anmeldung (kurs_id, mitglied_id)
        VALUES (?, ?)
    """, anmeldungen)

    conn.commit()

def show_data(conn):
    cursor = conn.cursor()

    print("\n--- FITNESSSTUDIO ---")
    for row in cursor.execute("SELECT * FROM fitnessstudio;"):
        print(row)

    print("\n--- TRAINER ---")
    for row in cursor.execute("SELECT * FROM trainer;"):
        print(row)

    print("\n--- MITGLIEDER ---")
    for row in cursor.execute("SELECT * FROM mitglied;"):
        print(row)

    print("\n--- KURSE ---")
    for row in cursor.execute("SELECT * FROM kurse;"):
        print(row)
```

```
    print("\n--- ANMELDUNGEN ---")
    for row in cursor.execute("SELECT * FROM anmeldung;"):
        print(row)


def main():
    conn = create_connection()
    try:
        create_tables(conn)
        insert_sample_data(conn)
        show_data(conn)
        print(f"\nDatenbank '{DB_NAME}' wurde erfolgreich erstellt und befüllt.")
    except sqlite3.Error as e:
        print("Fehler bei der Arbeit mit SQLite:", e)
    finally:
        conn.close()


if __name__ == "__main__":
    main()
```

Anvil Frontend Code

FROM 1

```
class Form1(Form1Template):
  def __init__(self, **properties):
    # Set Form properties and Data Bindings.
    self.init_components(**properties)


    # Any code you write here will run before the form opens.


  @handle("button_1", "click")
  def button_1_click(self, **event_args):
    """This method is called when the button is clicked"""
    open_form('Form2')
```

FORM 2

```
class Form2(Form2Template):
  def __init__(self, **properties):
```

Name: Muhammet Altuntas **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
    data = anvil.server.call('hol_alle_kurs_daten')

    print(data)
    self.repeating_panel_kurs.items = data

class RowTemplate1(RowTemplate1Template):
  def __init__(self, **properties):
    # Set Form properties and Data Bindings.
    self.init_components(**properties)

    # Any code you write here will run before the form opens.

  @handle("button_1", "click")
  def button_1_click(self, **event_args):
    """This method is called when the button is clicked"""
    open_form('Form3', kurs_id=self.item['kurs_id'])
```

From 3

```
class Form3(Form3Template):
  def __init__(self, **properties):
    self.init_components(**properties)

    self.kurse_laden()

  def kurse_laden(self):
    daten = anvil.server.call('hole_verfuegbare_mitglieder',self.item.kurs_id)


    print(daten)
```

Name: Muhammet Altuntas **Datum:** 08.04.2026 **Klasse:** 3AHIF

Anvil Backend Code

```
import anvil.tables as tables

import anvil.tables.query as q

from anvil.tables import app_tables

import anvil.files

from anvil.files import data_files

import anvil.server

import sqlite3


# This is a server module. It runs on the Anvil server,

# rather than in the user's browser.

#

# To allow anvil.server.call() to call functions here, we mark

# them with @anvil.server.callable.

# Here is an example - you can replace it with your own:

#

# @anvil.server.callable

# def say_hello(name):

#   print("Hello, " + name + "!")

#   return 42

#


@anvil.server.callable

def hol_alle_kurs_daten():

  with sqlite3.connect(data_files["ALTUNTAS_Muhammet_fitnessstudio.db"]) as conn:


    conn.row_factory = sqlite3.Row



    cursor = conn.cursor()


    query = """

    SELECT

    k.kurs_id,
```

```
    k.bezeichnung AS kurs,

    k.wochentag,
    k.uhrzeit,
    t.vorname || ' ' || t.nachname AS trainer,

    COUNT(a.anmelde_id) AS aktuelle_teilnehmer,
    k.max_teilnehmer

FROM kurse as k

 JOIN trainer t  ON k.trainer_id = t.trainer_id
LEFT JOIN anmeldung a ON k.kurs_id = a.kurs_id

GROUP BY k.kurs_id
 ORDER  BY  k.wochentag, k.uhrzeit"""

    cursor.execute(query)

    rows = cursor.fetchall()



    return [dict(row) for row in rows]



def hole_verfuegbare_mitglieder(kurs_id):
  with sqlite3.connect(data_files["Fittnes.db"]) as conn:
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    query = """
        SELECT
            m.mitglied_id,
             m.vorname,
```

```
        m.nachname
     FROM  mitglied  m

    WHERE m.mitglied_id NOT IN (
       SELECT a.mitglied_id
         FROM anmeldung a
       WHERE a.kurs_id = ?

    )
    ORDER BY m.nachname,
    m.vorname

    """

 cursor.execute(query, (kurs_id,))

 rows = cursor.fetchall()


 return [dict(row) for row in rows]
```